MAL4 Campaign Review & Approval -- Low-Level Design

Project: Mobile Architecture Lab 4 (MAL4)
Track: MarTech -- Campaign Review & Approval
Document Identifier: MAL4-LLD-v1.0
Version: 1.0
Status: Draft -- Internal Review Pending
Preparation Date: 2026-11-01
Prepared By: Architecture & Engineering

Table of Contents

MAL4 Campaign Review & Approval — Low-Level Design (LLD) 3

5.1 Document Control 3

5.2 Module Specifications 3

5.2.1 Authentication & Authorization 3

5.2.2 Campaign Management 6

5.2.3 Campaign Version Management 9

5.2.4 Review & Approval Workflow (Merged Module) 12

5.2.5 Audit Management 19

5.2.6 Notification Management 21

5.2.7 Scheduling 24

6. Critical Modules — Deep Dive 26

6.1 Campaign State Management 26

6.2 Campaign Version Management 29

6.3 Approval Workflow 29

6.4 Concurrent Modification 30

6.5 Audit Management 31

6.6 Scheduling 31

7. API and Contract Specifications 31

8. LLD Diagrams 32

MAL4 Campaign Review & Approval — Low-Level Design (LLD)

5.1 Document Control

Field	Value
Document Title	MAL4 Campaign Review & Approval — Low-Level Design
Document ID	MAL4-LLD-v1.0
Version	1.0
Status	Draft — pending team review
Author	Architecture & Engineering Lead
HLD Reference	`MAL4-HLD.md` (MAL4-HLD-v1.0) — this LLD derives every module below from that document's §4.9 Module Decomposition. No module appears here that is not in the approved HLD.
Approval Status	Review pending

Table 1

Revision History

Version	Date	Change
0.1	2026-11-01	Initial draft for internal review
1.0	2026-11-01	First complete release, covering all seven module specifications through test scenarios

Table 2

What this document does not claim: no Flutter/Dart code has been compiled, run, or tested. Every class signature, pseudocode block, and test scenario below is a design artifact, stated as such — not implementation evidence.

5.2 Module Specifications

Seven modules, matching HLD §4.9 exactly: Authentication & Authorization, Campaign Management, Campaign Version Management, Review & Approval Workflow, Audit Management, Notification Management, Scheduling. Campaign Management and Campaign Version Management share the same `Campaign` aggregate class — their specs cross-reference rather than duplicate class/data-structure detail.

5.2.1 Authentication & Authorization

A. Module Overview

- Module ID: MOD-AUTH
- Purpose: identify the current actor and role; gate every state-transitioning use case.
- Responsibilities: FR-017 (RBAC), FR-018 (self-review prevention), NFR-008/009/010.

- Scope: authorization checks only — does not implement a real Identity Provider.
- Dependencies: external Identity Provider (not implemented for MAL4).
- Preconditions: none (this module is the entry gate).
- Postconditions: an `AuthenticatedUser{id, role}` is available, or the calling use case receives `UnauthorizedError`/`AuthenticationError` before any domain mutation.

B. Internal Component Design

- `AuthProvider` (interface, in `application/ports/`) — implemented by `AuthProviderImpl` (in `infrastructure/auth/`, a mock/stub for MAL4).
- Pattern: Adapter (over the Identity Provider boundary), consumed via dependency injection by every use case.
- Dependency direction: `application/ports/auth_provider.dart` has no implementation detail; `infrastructure/auth/auth_provider_impl.dart` implements it.

C. Class Specifications

```
abstract class AuthProvider {
  Future<AuthenticatedUser> currentUser();
}

@immutable
class AuthenticatedUser {
  final String id;
  final Role role;
  const AuthenticatedUser({required this.id, required this.role});
}

enum Role { marketer, reviewer, admin }
```

D. Data Structures

`AuthenticatedUser` is a value object (`id`: non-null `String`, `role`: non-null `Role` enum). No nullability on either field — an unauthenticated state is represented by the *absence* of an `AuthenticatedUser` (a thrown `AuthenticationError`), never a null-role sentinel.

E. Algorithms and Processing Logic

Operation: Role-gate check (used identically by every use case)

```
Operation: authorize(requiredRole)
Input: requiredRole (Role), current AuthenticatedUser
Precondition: AuthProvider.currentUser() has resolved
Steps:
  1. user = await authProvider.currentUser()
  2. if user.role != requiredRole and user.role != Role.admin:
        return Err(UnauthorizedError())
  3. proceed with use case, passing user.id forward
Output: Result<AuthenticatedUser, UnauthorizedError>
Postcondition: no domain mutation has occurred if Err
Complexity: O(1)
```

F. Module Interfaces

Interface	Method	Params	Return	Exceptions	Preconditions	Postconditions	Calling Module	Consuming Module
`AuthProvider`	`currentUser()`	none	`Future<AuthenticatedUser>`	`AuthenticationError`	A session exists (even if mocked)	Returns identity/role or throws	All other modules	—

Table 3

G. Database Specifications

None persisted by this module — role assignment data ownership is deferred to a future Admin configuration capability, not yet designed.

H. Error Specifications

Error ID	Description	Trigger	Detection Point	Handling	Recovery	User Message
ERR-AUTH-01	`AuthenticationError`	No valid session	`AuthProvider.currentUser()`	Use case returns `Result` failure	User re-authenticates (out of scope to implement)	"Please sign in again."
ERR-AUTH-02	`UnauthorizedError`	Role mismatch	Use case's authorize step	`Result` failure, no mutation	None — not a recoverable state, a permission boundary	"You don't have permission to do this."

Table 4

I. Input/Output Specifications

Input	Valid	Invalid	Output
`requiredRole: Role.reviewer`, actual role `Role.marketer`	—	Mismatch	`Err(UnauthorizedError())`
`requiredRole: Role.marketer`, actual role `Role.marketer`	Match	—	`Ok(AuthenticatedUser)`
No session	—	Always invalid	`Err(AuthenticationError())`

Table 5

J. Dependencies

Internal: none (this is a leaf infrastructure module). External: Identity Provider (not implemented). Configuration: none established. Failure behavior: see H.

K. Test Specifications

Test ID	Scenario	Input	Precondition	Expected Result
TC-AUTH-01	Wrong-role rejection	Marketer attempts a Reviewer-only action	Authenticated as Marketer	`Err(UnauthorizedError)`, no repository call made
TC-AUTH-02	No session	Any action, no `AuthProvider` session	None	`Err(AuthenticationError)`

Table 6

5.2.2 Campaign Management

A. Module Overview

- Module ID: MOD-CAMPAIGN
- Purpose: create and edit campaigns while in Draft or Changes Requested state.
- Responsibilities: FR-001 (creation), FR-002 (editing rules), FR-003 (soft delete, Could-priority, not detailed further — no FR-level guidance beyond "archive, don't hard-delete").
- Scope: does not include version creation itself (owned by 5.2.3) or review decisions (5.2.4).
- Dependencies: Campaign Version Management (shares the aggregate), Authentication & Authorization.
- Preconditions: authenticated Marketer.
- Postconditions: a `Campaign` exists in Draft state (creation) or its working draft fields are updated (edit).

B. Internal Component Design

- `CreateCampaign`, `EditCampaign` use cases (Application layer) delegate to the `Campaign` aggregate (Domain layer, full spec in 5.2.3 — shared class).
- Pattern: Command (use case as command handler) + Repository (persistence seam).

C. Class Specifications

```
class CreateCampaign {
  final CampaignRepository _campaigns;
  final AuditLogRepository _audit;
  final AuthProvider _auth;
  CreateCampaign(this._campaigns, this._audit, this._auth);

  Future<Result<Campaign, DomainError>> execute(CreateCampaignCommand command) async {
    final user = await _auth.currentUser();
    if (user.role != Role.marketer) return Err(UnauthorizedError());
    _validateRequiredFields(command); // FR-001 table: name, objective, audience, budget,
        schedule required
    final campaign = Campaign.create(id: command.id, submitterId: user.id);
    await _campaigns.save(campaign, expectedVersion: 0);
    await _audit.append(AuditEntry(/* action: "created" */));
    return Ok(campaign);
  }
}

class EditCampaign {
```

```
  final CampaignRepository _campaigns;
  final AuthProvider _auth;
  EditCampaign(this._campaigns, this._auth);

  Future<Result<Campaign, DomainError>> execute(EditCampaignCommand command) async {
    final user = await _auth.currentUser();
    final campaign = await _campaigns.load(command.campaignId);
    if (campaign.state == CampaignState.approved) {
      // FR-014: editing an Approved campaign is a material change, not a plain edit
      campaign.invalidateApprovalOnMaterialChange();
    } else if (![CampaignState.draft, CampaignState.changesRequested].contains(campaign.state)) {
      return Err(DomainError.invalidTransition('Cannot edit in state ${campaign.state}'));
    }
    campaign.applyFieldEdits(command.fields); // mutates the unversioned working draft only
    final saveResult = await _campaigns.save(campaign, expectedVersion: campaign.currentVersion);
    if (saveResult is Err) return Err(DomainError.concurrentModification());
    return Ok(campaign);
  }
}
```

D. Data Structures

`CreateCampaignCommand { id: String, name: String, objective: String, targetAudience: String, budget: double, schedule: DateTimeRange, channels: List<String>?, contentReferences: List<String>? }` — `channels`/`contentReferences` nullable (Should-priority per FR-001's table, not Must); all other fields non-null, validated before `Campaign.create()`. `EditCampaignCommand { campaignId: String, fields: CampaignFieldEdits }` — `CampaignFieldEdits` mirrors the same fields, all nullable (partial edits allowed, unset fields unchanged).

E. Algorithms and Processing Logic

Operation: Campaign creation field validation (FR-001)

```
Operation: validateRequiredFields(command)
Input: CreateCampaignCommand
Precondition: none
Steps:
  1. for field in [name, objective, targetAudience, budget, schedule]:
       if field is null or (field is String and field.trim().isEmpty):
         raise ValidationError(field)
  2. if budget <= 0: raise ValidationError('budget')
  3. if schedule.start >= schedule.end: raise ValidationError('schedule')
Output: void, or throws ValidationError
Postcondition: a complete, valid snapshot is ready to pass to Campaign.create()
Complexity: O(1) — fixed field count
```

F. Module Interfaces

Interface	Method	Params	Return	Exceptions	Calling Module	Consuming Module
`CreateCampaign`	`execute(CreateCampaignCommand)`	command	`Future<Result<Campaign, DomainError>>`	none (typed Result)	Presentation	—
`EditCampaign`	`execute(EditCampaignCommand)`	command	`Future<Result<Campaign, DomainError>>`	none (typed Result)	Presentation	—

Table 7

G. Database Specifications

Entity-level constraints only (physical schema is the separately maintained data-design documentation, not invented here): a `campaigns` table concept with one row per aggregate (id PK, state, current_version, submitter_id) — see 5.2.3.G for the full entity-relationship constraint set shared across modules.

H. Error Specifications

Error ID	Description	Trigger	Detection	Handling	User Message
ERR-CAMP-01	`ValidationError`	Missing/invalid required field at creation	`validateRequiredFields`	Rejected before `Campaign.create()` is even called	"Please complete all required fields."
ERR-CAMP-02	`InvalidTransitionError`	Edit attempted on Pending Review/ Scheduled/Rejected	`Campaign` aggregate state guard	`Result` failure	"This campaign can't be edited right now."

Table 8

I. Input/Output Specifications

Scenario	Input	Output
Valid creation	All required fields present, budget > 0, schedule.start < schedule.end	`Ok(Campaign)` in Draft state
Missing objective	`objective: ""`	`Err(ValidationError('objective'))`
Edit on Pending Review	`campaign.state == pendingReview`	`Err(DomainError.invalidTransition(...))`
Edit on Approved (material change)	`campaign.state == approved`	`Ok(Campaign)` in Draft state, with the prior approval superseded

Table 9

J. Dependencies

Internal: `Campaign` aggregate (5.2.3), `CampaignRepository`, `AuditLogRepository`, `AuthProvider`. External: none. Failure behavior: see H.

K. Test Specifications

Test ID	Scenario	Input	Precondition	Expected Result
TC-CAMP-01	Valid creation	Complete, valid fields	Authenticated Marketer	`Ok(Campaign)`, state = Draft
TC-CAMP-02	Missing required field	`budget: null`	—	`Err(ValidationError)`
TC-CAMP-03	Edit rejected in Pending Review	Edit attempt	`campaign.state == pendingReview`	`Err(InvalidTransitionError)`
TC-CAMP-04	Material-change path	Edit attempt	`campaign.state == approved`	State → Draft, approval superseded, edit applied

Table 10

5.2.3 Campaign Version Management

A. Module Overview

- Module ID: MOD-VERSION
- Purpose: manage immutable `CampaignVersion` snapshots and the optimistic-concurrency token (`currentVersion`).
- Responsibilities: FR-013 (versioning), NFR-001/003 (consistency/concurrency).
- Scope: shares the `Campaign` aggregate class with Campaign Management (5.2.2) — this spec documents the version-specific methods and data shapes; aggregate-wide detail (state enum, full method list) is not re-derived here, only cross-referenced.
- Dependencies: none beyond the `Campaign` aggregate itself.
- Preconditions: a `Campaign` exists.
- Postconditions: a new `CampaignVersion` is appended, `currentVersion` incremented, exactly at submit/revise-resubmit — never on a plain Draft edit (5.2.2).

B. Internal Component Design

Pattern: the `Campaign` aggregate root owns an append-only `List<CampaignVersion>` — no separate "VersionManager" class exists; version creation is a side effect of `Campaign.submit()`/`reviseAndResubmit()`, never a standalone operation callable out of context.

C. Class Specifications

```
@immutable
class CampaignVersion {
  final int versionNumber;
  final String name;
  final String objective;
  final String targetAudience;
  final double budget;
  final DateTimeRange schedule;
  final List<String> channels;
  final List<String> contentReferences;
  final DateTime createdAt;

  const CampaignVersion({
    required this.versionNumber, required this.name, required this.objective,
    required this.targetAudience, required this.budget, required this.schedule,
    required this.channels, required this.contentReferences, required this.createdAt,
  });
  // No copyWith, no setter: a "change" always produces a new CampaignVersion via
  // Campaign.submit()/reviseAndResubmit() — never an in-place mutation (A-010).
}
```

Full `Campaign` aggregate class (owning `_versions`, `_currentVersion`): see 5.2.4.C (documented once, alongside Review & Approval Workflow, since that module's methods are what mutate version state most visibly) — not duplicated here.

D. Data Structures

All fields in `CampaignVersion` are non-null (a version is either a complete snapshot or it does not exist — there is no partial/draft version). `versionNumber` is a positive, strictly-increasing `int`, assigned as `currentVersion + 1` at creation, never reused

even if a campaign is later restarted from Rejected.

E. Algorithms and Processing Logic

Operation: Version creation (FR-013)

```
Operation: createVersion(campaign, snapshot)
Input: campaign (Campaign, in Draft or ChangesRequested state), snapshot (validated field set)
Precondition: campaign.state in {Draft, ChangesRequested}
Steps:
  1. newVersionNumber = campaign.currentVersion + 1
  2. version = CampaignVersion(versionNumber: newVersionNumber, ...snapshot, createdAt: now())
  3. campaign.versions.append(version)          // append-only; no prior version is touched
  4. campaign.currentVersion = newVersionNumber
  5. campaign.state = PendingReview
Output: campaign (mutated in place, caller persists via repository)
Postcondition: exactly one new version exists; all prior versions are unchanged
Complexity: O(1) amortized append
```

F. Module Interfaces

No standalone interface — version creation is reached only through `Campaign.submit()`/`Campaign.reviseAndResubmit()` (5.2.4.F documents those as part of the Review & Approval Workflow's use-case interfaces, since submission is the entry point).

G. Database Specifications

Entity-level constraints (not the physical schema — the separately maintained data-design documentation owns that):

Concept	Constraint
`campaigns`	One row per aggregate; `current_version` column is the concurrency token compared on every write
`campaign_versions`	Append-only; foreign-keyed to `campaigns`; `version_number` unique per campaign, strictly increasing; no `UPDATE`/`DELETE` statement should ever be issued against this table by any repository method
`approval_records`	Foreign-keyed to a specific `(campaign_id, version_number)` pair
`audit_log`	Foreign-keyed to `campaigns`, indexed by campaign/actor/timestamp

Table 11

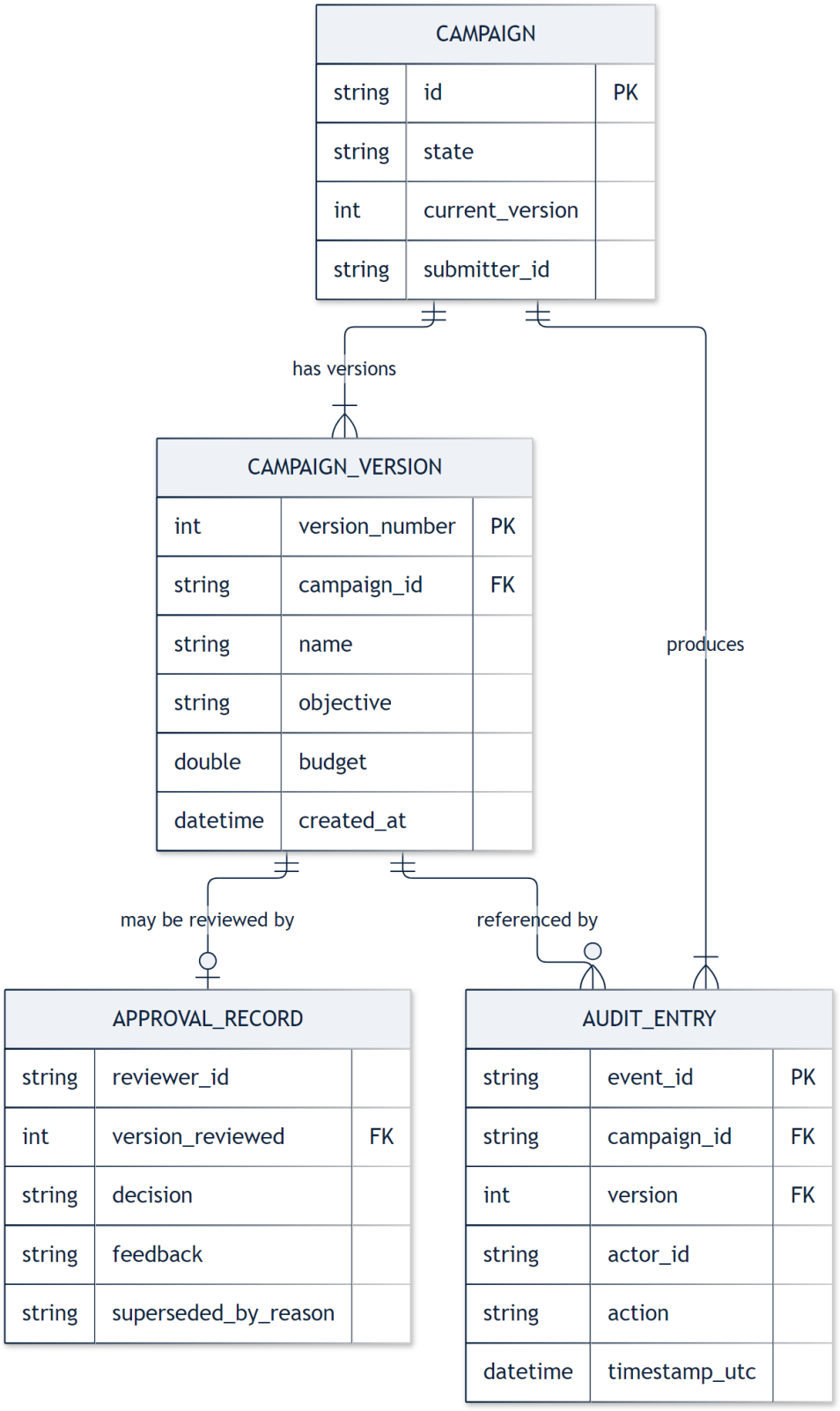


Figure 1: *Conceptual Entity Relationship Diagram*

H. Error Specifications

Error ID	Description	Trigger	Detection	Handling	User Message
ERR-VER-01	`ConcurrentModificationError`	`save()` with a stale `expectedVersion`	`CampaignRepository.save()` `Result` failure, write rejected	"This campaign changed since you loaded it — please reload."

Table 12

I. Input/Output Specifications

Scenario	Input	Output
First submission	`campaign.currentVersion == 0`, `state == draft`	New version 1, `state == pendingReview`
Revision resubmission	`campaign.currentVersion == 2`, `state == changesRequested`	New version 3, `state == pendingReview`
Stale save	`expectedVersion: 1`, stored version already 2	`Err(ConcurrentModificationError)`

Table 13

J. Dependencies

Internal: `Campaign` aggregate. External: none. Configuration: none.

K. Test Specifications

Test ID	Scenario	Expected Result
TC-VER-01	Submit from Draft	Version 1 created, prior (none) untouched
TC-VER-02	Revise from Changes Requested	New version appended, old versions still readable and unchanged
TC-VER-03	Stale write	`ConcurrentModificationError`

Table 14

5.2.4 Review & Approval Workflow (Merged Module)

A. Module Overview

- Module ID: MOD-REVIEW-APPROVAL
- Purpose: let an authorized Reviewer evaluate the current version of a Pending-Review campaign and record approve/reject/request-changes; handle re-approval invalidation on material change.
- Responsibilities: FR-006 (review action), FR-007 (approval), FR-008 (rejection), FR-009 (request changes), FR-010 (revision trigger), FR-011 (feedback visibility), FR-014 (material-change invalidation, executed by 5.2.2 but whose binding rule is owned here).
- Scope: this is the module containing the full `Campaign` aggregate class listing (shared by 5.2.2/5.2.3 by reference).

- Dependencies: Campaign Version Management (5.2.3), Authentication & Authorization (5.2.1), Audit Management (5.2.5).
- Preconditions: campaign in Pending Review; reviewer authenticated and not the submitter.
- Postconditions: an immutable `ApprovalRecord` exists, bound to the version reviewed; campaign transitions to Approved/Rejected/Changes Requested.

B. Internal Component Design

- `ReviewCampaign`, `ReviseCampaign` use cases (Application).
- `Campaign` aggregate (Domain) — full class below.
- Design patterns: Command (use cases), Aggregate Root (DDD — `Campaign` is the sole consistency boundary), Dependency Inversion (repository/adapter interfaces owned in `application/ports/`).
- Dependency direction: Application → Domain only; Domain has zero outward dependencies.

C. Class Specifications

```
enum CampaignState { draft, pendingReview, changesRequested, approved, rejected, scheduled }
enum ReviewDecision { approve, reject, requestChanges }

class Campaign {
  final String id;
  CampaignState _state;
  int _currentVersion;
  final List<CampaignVersion> _versions;
  ApprovalRecord? _activeApproval;
  final String _submitterId;

  CampaignState get state => _state;
  int get currentVersion => _currentVersion;
  ApprovalRecord? get activeApproval => _activeApproval;

  Campaign._(this.id, this._state, this._currentVersion, this._versions, this._activeApproval,
      this._submitterId);
  factory Campaign.create({required String id, required String submitterId}) =>
      Campaign._(id, CampaignState.draft, 0, const [], null, submitterId);

  Campaign submit(CampaignVersionDraft snapshot) {
    _requireState({CampaignState.draft, CampaignState.changesRequested});
    final version = _currentVersion + 1;
    _versions.add(snapshot.toVersion(version));
    _currentVersion = version;
    _state = CampaignState.pendingReview;
    return this;
  }

  Campaign withdraw() {
    _requireState({CampaignState.pendingReview});
    if (_activeApproval != null && _activeApproval!.versionReviewed == _currentVersion) {
      throw InvalidTransitionError('A decision already exists for this version');
    }
    _state = CampaignState.draft;
    return this;
  }

  Campaign approve(String reviewerId, int versionBeingReviewed) {
    _requireState({CampaignState.pendingReview});
    _requireCurrentVersion(versionBeingReviewed);
    _requireNotSelfReview(reviewerId);
    _activeApproval = ApprovalRecord(
      reviewerId: reviewerId, decision: ReviewDecision.approve,
```

```
      timestamp: DateTime.now().toUtc(), versionReviewed: _currentVersion);
    _state = CampaignState.approved;
    return this;
  }

  Campaign reject(String reviewerId, int versionBeingReviewed, String reason) {
    _requireState({CampaignState.pendingReview});
    _requireCurrentVersion(versionBeingReviewed);
    _requireNotSelfReview(reviewerId);
    if (reason.trim().isEmpty) throw ArgumentError('Rejection reason is required (FR-008)');
    _state = CampaignState.rejected;
    return this;
  }

  Campaign requestChanges(String reviewerId, int versionBeingReviewed, String feedback) {
    _requireState({CampaignState.pendingReview});
    _requireCurrentVersion(versionBeingReviewed);
    _requireNotSelfReview(reviewerId);
    if (feedback.trim().isEmpty) throw ArgumentError('Feedback is required (FR-009)');
    _state = CampaignState.changesRequested;
    return this;
  }

  Campaign reviseAndResubmit(CampaignVersionDraft snapshot) {
    _requireState({CampaignState.changesRequested});
    return submit(snapshot);
  }

  Campaign invalidateApprovalOnMaterialChange() {
    _requireState({CampaignState.approved});
    _state = CampaignState.draft; // activeApproval retained, marked superseded (not deleted)
        for audit
    return this;
  }

  Campaign schedule() {
    _requireState({CampaignState.approved});
    if (_activeApproval == null || _activeApproval!.versionReviewed != _currentVersion) {
      throw InvalidTransitionError('Approved version is no longer current');
    }
    _state = CampaignState.scheduled;
    return this;
  }

  void _requireState(Set<CampaignState> allowed) {
    if (!allowed.contains(_state)) throw InvalidTransitionError('Cannot act on $this from
        state $_state');
  }
  void _requireCurrentVersion(int v) {
    if (v != _currentVersion) throw StaleReviewError(attempted: v, current: _currentVersion);
  }
  void _requireNotSelfReview(String reviewerId) {
    if (reviewerId == _submitterId) throw SelfReviewError(reviewerId);
  }
}

@immutable
class ApprovalRecord {
  final String reviewerId;
  final ReviewDecision decision;
  final DateTime timestamp;
  final int versionReviewed;
  final String? feedback;
  final String? supersededByReason;
  const ApprovalRecord({required this.reviewerId, required this.decision, required
      this.timestamp,
      required this.versionReviewed, this.feedback, this.supersededByReason});
}

class ReviewCampaign {
```

```
  final CampaignRepository _campaigns;
  final AuditLogRepository _audit;
  final AuthProvider _auth;
  final void Function(DomainEvent) _emit;
  ReviewCampaign(this._campaigns, this._audit, this._auth, this._emit);

  Future<Result<Campaign, DomainError>> execute(ReviewCommand command) async {
    final user = await _auth.currentUser();
    if (user.role != Role.reviewer) return Err(UnauthorizedError());
    final campaign = await _campaigns.load(command.campaignId); // always fresh — never cached
    final expectedVersion = campaign.currentVersion;
    try {
      switch (command.decision) {
        case ReviewDecision.approve: campaign.approve(user.id, command.versionBeingReviewed);
        case ReviewDecision.reject: campaign.reject(user.id,
            command.versionBeingReviewed, command.reason!);
        case ReviewDecision.requestChanges: campaign.requestChanges(user.id,
            command.versionBeingReviewed, command.feedback!);
      }
    } on StaleReviewError catch (e) {
      return Err(DomainError.staleReview(attempted: e.attempted, current: e.current));
    } on SelfReviewError {
      return Err(DomainError.selfReview());
    } on InvalidTransitionError catch (e) {
      return Err(DomainError.invalidTransition(e.message));
    }
    final saveResult = await _campaigns.save(campaign, expectedVersion: expectedVersion);
    if (saveResult is Err) return Err(DomainError.concurrentModification());
    await _audit.append(AuditEntry(/* action matching command.decision */));
    _emit(_eventFor(command.decision, campaign));
    return Ok(campaign);
  }
}
```

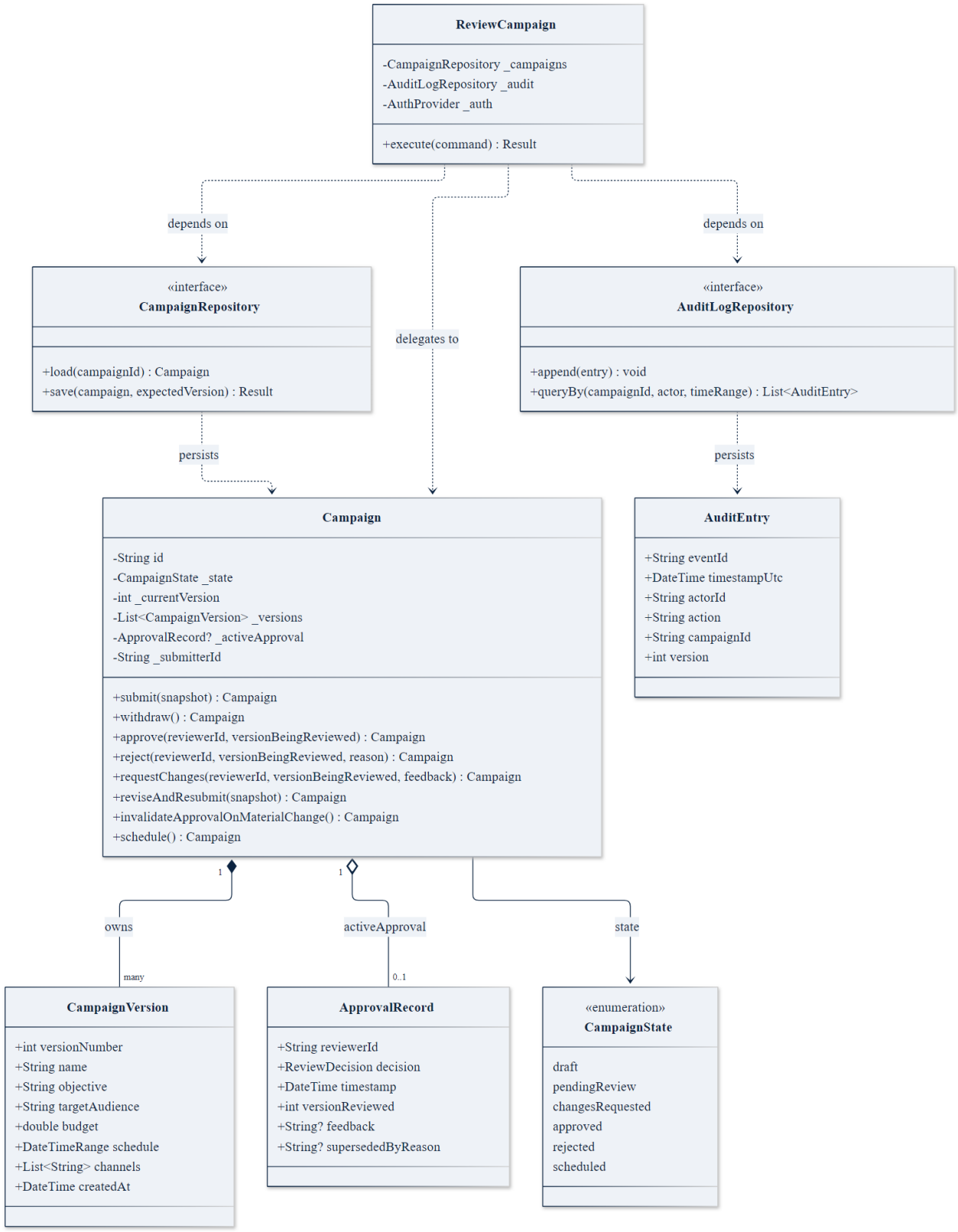


Figure 2: *UML Class Diagram*

D. Data Structures

```
class ReviewCommand {
  final String campaignId;
  final int versionBeingReviewed;   // caller's believed-current version — used only to DETECT
      staleness, never to RECORD the binding
```

```
  final ReviewDecision decision;
  final String? reason;             // required iff decision == reject
  final String? feedback;           // required iff decision == requestChanges
  ReviewCommand({required this.campaignId, required this.versionBeingReviewed, required
      this.decision, this.reason, this.feedback});
}
```

`reason`/`feedback` nullable at the type level but validated as required at the domain layer for their respective decisions (ArgumentError if empty) — not re-validated redundantly in the command object's constructor, to keep validation in one place (the aggregate).

E. Algorithms and Processing Logic

Operation: Approval with version-binding (FR-007, NFR-002) — the authoritative pseudocode for this project's single most scrutinized invariant:

```
Operation: approve(campaign, reviewerId, versionBeingReviewed)
Input: campaign (Campaign, freshly loaded), reviewerId (String), versionBeingReviewed (int,
    caller-supplied)
Precondition: campaign.state == PendingReview
Steps:
  1. if campaign.state != PendingReview: raise InvalidTransitionError
  2. if versionBeingReviewed != campaign.currentVersion:
       raise StaleReviewError(attempted: versionBeingReviewed, current: campaign.currentVersion)
  3. if reviewerId == campaign.submitterId: raise SelfReviewError(reviewerId)
  4. campaign.activeApproval = ApprovalRecord(
         reviewerId: reviewerId, decision: Approve,
         versionReviewed: campaign.currentVersion,   // the aggregate's OWN value, never
             the caller's claimed value
         timestamp: now())
  5. campaign.state = Approved
Output: campaign (mutated)
Postcondition: exactly one ApprovalRecord exists, bound to the version that was actually current
    at decision time
Error conditions: InvalidTransitionError, StaleReviewError, SelfReviewError
Complexity: O(1)
```

Note the deliberate detail in step 4: `versionReviewed` is set from the aggregate's own `currentVersion`, not the caller-supplied `versionBeingReviewed` — the caller's value is used only to *detect* staleness in step 2, never to *record* the binding. This prevents a stale caller value from leaking into the persisted record.

F. Module Interfaces

Interface	Method	Parameters	Return	Exceptions	Preconditions	Postconditions	Calling Module	Consuming Module
`ReviewCampaign`	`execute(ReviewCommand)`	command	`Future<Result<Campaign, DomainError>>`	none (typed `Result`)	Authenticated Reviewer	`ApprovalRecord` + audit entry committed, or no mutation	Presentation	—
`ReviseCampaign`	`execute(ReviseCampaignCommand)`	command	`Future<Result<Campaign, DomainError>>`	none	Campaign in Changes Requested	New version + audit entry, or no mutation	Presentation	—

Table 15

G. Database Specifications

See 5.2.3.G — `approval_records` table concept (foreign-keyed to `(campaign_id, version_number)`, nullable `superseded_by_reason` column). Not re-derived here.

H. Error Specifications

Error ID	Description	Trigger	Detection	Handling	Recovery	User Message
ERR-REV-01	`StaleReviewError`	`versionBeingReviewed != currentVersion`	`Campaign.approve/reject/requestChanges`	`Result` failure	Reviewer reloads the current version	"This campaign has changed since you opened it — please review the latest version."
ERR-REV-02	`SelfReviewError`	`reviewerId == submitterId`	Same methods	`Result` failure	Not recoverable — a different reviewer must act	"You can't review a campaign you submitted."
ERR-REV-03	`InvalidTransitionError` (duplicate decision)	Second decision on an already-decided campaign	`_requireState` guard	`Result` failure	None needed — first decision stands	"This campaign has already been decided."

Table 16

I. Input/Output Specifications

Scenario	Input	Output
Valid approval	`versionBeingReviewed == currentVersion`, reviewer != submitter	`Ok(Campaign)`, state = Approved, `ApprovalRecord` created
Stale review	`versionBeingReviewed = 1`, `currentVersion = 2`	`Err(DomainError.staleReview(attempted: 1, current: 2))`
Self-review attempt	`reviewerId == submitterId`	`Err(DomainError.selfReview())`
Reject without reason	`reason: ""`	`ArgumentError` (surfaced as a validation failure before persistence)

Table 17

J. Dependencies

Internal: `Campaign` aggregate, `CampaignRepository`, `AuditLogRepository`, `AuthProvider`, Domain Event Bus (for `NotificationGateway` dispatch, 5.2.6). External: none. Failure behavior: see H.

K. Test Specifications

Test ID	Scenario	Input	Precondition	Expected Result
TC-REV-01	Valid approval	Current version, non-submitter reviewer	Pending Review	`Ok`, Approved, `ApprovalRecord` bound to current version
TC-REV-02	Stale review (full scenario)	Submit v1 → withdraw → resubmit v2 → approve with `VersionBeingReviewed=1`	See §6.1 scenario	`Err(StaleReviewError(1, 2))`
TC-REV-03	Self-review	`reviewerId == submitterId`	Pending Review	`Err(SelfReviewError)`
TC-REV-04	Rejection without reason	`reason: ""`	Pending Review	`ArgumentError` before any state mutation
TC-REV-05	Material-change invalidation	Edit an Approved campaign	Approved, `activeApproval` set	State → Draft, prior `ApprovalRecord` preserved with `supersededByReason` set

Table 18

5.2.5 Audit Management

A. Module Overview

- Module ID: MOD-AUDIT
- Purpose: capture an immutable, queryable record of every significant lifecycle action.
- Responsibilities: FR-015 (logging fields), FR-016 (history view), NFR-012 (immutability), NFR-013 (retention — provisional, not confirmed).
- Scope: append and query only — no update/delete capability exists anywhere in this module's interface, by construction.
- Dependencies: `AuditLogRepository`.
- Preconditions: a triggering lifecycle action has occurred (created/submitted/approved/rejected/changes_requested/revised/scheduled).
- Postconditions: exactly one `AuditEntry` exists per triggering action, committed in the same local transaction as the state change it records.

B. Internal Component Design

`AuditEntry` (Domain, immutable value object) + `AuditLogRepository` (interface in `application/ports/`, implemented in `infrastructure/persistence/`). No "AuditService" class exists separately — every use case calls `AuditLogRepository.append()` directly as its final persistence step, keeping the audit write inside the same atomic transaction as the state write (5.2.3.G).

C. Class Specifications

```
@immutable
```

```
class AuditEntry {
  final String eventId;
  final DateTime timestampUtc;
  final String actorId;
  final String action; // "created" | "submitted" | "approved" | "rejected" |
      "changes_requested" | "revised" | "scheduled"
  final String campaignId;
  final int version;
  final String? details;
  const AuditEntry({required this.eventId, required this.timestampUtc, required this.actorId,
      required this.action, required this.campaignId, required this.version, this.details});
  // Deliberately no toJson-driven update method anywhere in this class or its repository.
}

abstract class AuditLogRepository {
  Future<void> append(AuditEntry entry);
  Future<List<AuditEntry>> queryBy({String? campaignId, String? actor, DateTimeRange?
      timeRange});
  // No update(), no delete() — omission is structural, not an oversight.
}
```

D. Data Structures

All fields non-null except `details` (optional free-text context, e.g. rejection reason or feedback). `action` is a closed string enum in practice (the seven values listed), validated at the point each use case constructs the entry — not an open string field in intent, even though typed as `String` for serialization simplicity.

E. Algorithms and Processing Logic

Operation: Audit event creation rule (applies identically across every use case)

```
Operation: recordAuditEntry(action, actor, campaign, details?)
Input: action (one of the 7 closed values), actor (AuthenticatedUser), campaign (post-mutation
    Campaign), details (optional)
Precondition: the triggering state mutation has already succeeded against the in-memory aggregate
Steps:
  1. entry = AuditEntry(
       eventId: generateId(), timestampUtc: now().toUtc(), actorId: actor.id,
       action: action, campaignId: campaign.id, version: campaign.currentVersion, details:
           details)
  2. include entry's insert in the SAME database transaction as the campaign-state write
     (never a separate, subsequent transaction — see 5.2.3.G)
Output: void
Postcondition: the audit entry and the state it describes either both exist or neither does
Complexity: O(1) write; O(log n) for indexed queryBy() reads
```

F. Module Interfaces

Interface	Method	Parameters	Return	Exceptions	Calling Module	Consuming Module
`AuditLogRepository`	`append(AuditEntry)`	entry	`Future<void>`	`PersistenceUnavailableError`	Every action-performing module	—
`AuditLogRepository`	`queryBy({campaignId?, actor?, timeRange?})`	filters	`Future<List<AuditEntry>>`	`PersistenceUnavailableError`	Presentation (history view, FR-016)	—

Table 19

G. Database Specifications

`audit_log` table concept: append-only, indexed on `campaign_id`, `actor_id`, `timestamp_utc` (to satisfy FR-015's query requirement). No UPDATE/DELETE SQL statement should ever be generated by any repository method against this table — physical column types/lengths belong to the separately maintained data-design documentation.

H. Error Specifications

Error ID	Description	Trigger	Handling
ERR-AUDIT-01	`PersistenceUnavailableError`	Local DB write failure during the atomic state+audit transaction	Entire transaction rolled back — the campaign state write is also undone, since they're atomic (5.2.3.G)

Table 20

I. Input/Output Specifications

Scenario	Input	Output
Successful approval	`action: "approved"`, valid campaign/actor	One new `AuditEntry` row, committed with the state change
Query by campaign	`campaignId: "c-123"`	All entries for that campaign, chronologically orderable by `timestampUtc`
Attempted mutation	(no such API exists)	N/A — there is no code path to test for "rejecting an update," because no update method exists

Table 21

J. Dependencies

Internal: none beyond the repository interface. External: none. Failure behavior: see H — audit failure is never isolated from the state-change failure it would otherwise silently decouple from.

K. Test Specifications

Test ID	Scenario	Expected Result
TC-AUDIT-01	Append on every successful transition	One `AuditEntry` per transition, atomic with the state write
TC-AUDIT-02	Query by campaign/actor/time range	Correct filtered results
TC-AUDIT-03	API-surface assertion	No `update`/`delete` method exists on `AuditLogRepository` (compile-time check, not a runtime test)

Table 22

5.2.6 Notification Management

A. Module Overview

- Module ID: MOD-NOTIFY

- Purpose: best-effort delivery of lifecycle notifications, fully decoupled from the durable write path.
- Responsibilities: FR-019 (notification table), NFR-005 (must never block or roll back a state transition).
- Scope: adapter only — does not implement a real Notification Service.
- Dependencies: external Notification Service (not implemented); Domain Event Bus (in-process).
- Preconditions: a domain event has been emitted (always *after* the triggering transaction commits).
- Postconditions: delivery attempted; success or failure is never visible to the use case that emitted the event.

B. Internal Component Design

`NotificationGateway` (interface) + `NotificationGatewayImpl` (infrastructure adapter) subscribes to the in-process Domain Event Bus. Pattern: Observer (event subscription) + Adapter (external boundary).

C. Class Specifications

```
abstract class NotificationGateway {
  void notify(DomainEvent event); // fire-and-forget; return type is void, not
      Future<Result<...>>, by design
}

sealed class DomainEvent {
  final String campaignId;
  final int version;
  const DomainEvent(this.campaignId, this.version);
}
class CampaignSubmitted extends DomainEvent { const CampaignSubmitted(super.campaignId,
    super.version); }
class CampaignApproved extends DomainEvent { const CampaignApproved(super.campaignId,
    super.version); }
class CampaignRejected extends DomainEvent { const CampaignRejected(super.campaignId,
    super.version); }
class ChangesRequested extends DomainEvent { const ChangesRequested(super.campaignId,
    super.version); }
```

D. Data Structures

`DomainEvent` is a sealed class hierarchy — the compiler enforces exhaustive handling in `NotificationGatewayImpl`'s dispatch switch, so a new event type added later cannot be silently un-handled.

E. Algorithms and Processing Logic

Operation: Best-effort notification dispatch (NFR-005)

```
Operation: notify(event)
Input: event (DomainEvent), emitted strictly after the triggering transaction committed
Precondition: none — this is intentionally called unconditionally after every successful use case
Steps:
  1. try:
       deliver(event)  // calls the external Notification Service, not implemented for MAL4
     catch (deliveryError):
       log(deliveryError)
       scheduleRetry(event)  // exact backoff/retry-count policy: OPEN DECISION, not finalized
  2. return (void) -- regardless of success or failure of step 1
Output: none visible to the caller
Postcondition: the triggering use case's already-returned Result is never altered by this
```

```
    operation
Error conditions: delivery failures are caught and never re-raised to the caller
```

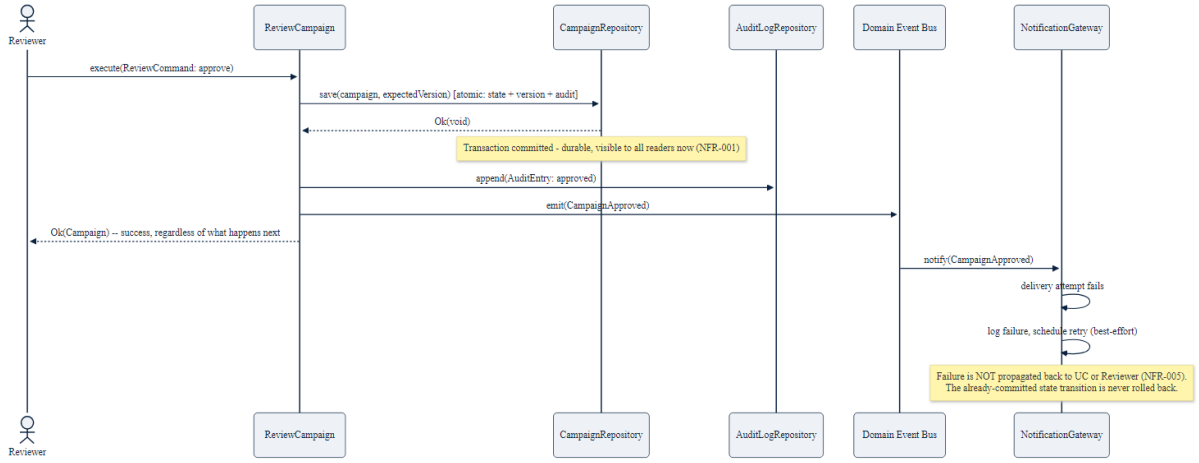


Figure 3: *Error Recovery Sequence*

F. Module Interfaces

Interface	Method	Parameters	Return	Exceptions	Calling Module	Consuming Module
`NotificationGateway`	`notify(DomainEvent)`	event	`void`	**none ever propagated**	Review & Approval Workflow, Campaign Management	—

Table 23

G. Database Specifications

None persisted in the current scope. **Open implementation detail:** a retry/outbox table may be needed for reliable best-effort delivery across app restarts — not yet designed, flagged rather than assumed.

H. Error Specifications

Error ID	Description	Trigger	Handling
ERR-NOTIFY-01	Delivery failure (any cause)	External Notification Service unreachable (not implemented, so always fails in the current build)	Logged, retry scheduled, never propagated to the triggering use case

Table 24

I. Input/Output Specifications

Scenario	Input	Output
Normal dispatch	`CampaignApproved(id, version)`	Attempted delivery; caller's `Result` from `ReviewCampaign.execute()` already returned, unaffected
Delivery failure	Same event, Notification Service throws	Logged + retry scheduled; still no effect on the already-returned `Result`

Table 25

J. Dependencies

Internal: Domain Event Bus. External: Notification Service (not implemented). Failure behavior: fully absorbed internally (see H) — this is the one module in the system whose errors are never surfaced upward.

K. Test Specifications

Test ID	Scenario	Expected Result
TC-NOTIFY-01	Notification failure isolation	Mock gateway throws; assert `ReviewCampaign.execute()` still returns `Ok` and the campaign state is persisted

Table 26

5.2.7 Scheduling

A. Module Overview

- Module ID: MOD-SCHED
- Purpose: enforce the scheduling gate — only an Approved campaign, still on its approved version, may transition to Scheduled.
- Responsibilities: FR-012 (gate), NFR-007 (duplicate-scheduling prevention).
- Scope: gate only — does not implement actual delivery/execution (explicitly out of scope, `MAL4-HLD.md` §4.3).
- Dependencies: Campaign Version Management (5.2.3), Audit Management (5.2.5).
- Preconditions: campaign in Approved state.
- Postconditions: campaign transitions to Scheduled, with an audit entry recording who/when/which version.

B. Internal Component Design

`ScheduleCampaign` use case → `Campaign.schedule()` (Domain, see 5.2.4.C for the full class — the `schedule()` method is listed there since it lives on the same aggregate).

C. Class Specifications

```
class ScheduleCampaign {
  final CampaignRepository _campaigns;
  final AuditLogRepository _audit;
  final AuthProvider _auth;
  ScheduleCampaign(this._campaigns, this._audit, this._auth);

  Future<Result<Campaign, DomainError>> execute(ScheduleCampaignCommand command) async {
    final user = await _auth.currentUser();
    final campaign = await _campaigns.load(command.campaignId); // fresh load, re-verifies
        approved version is still current
    final expectedVersion = campaign.currentVersion;
    try {
      campaign.schedule();
    } on InvalidTransitionError catch (e) {
      return Err(DomainError.invalidTransition(e.message));
```

```
    }
    final saveResult = await _campaigns.save(campaign, expectedVersion: expectedVersion);
    if (saveResult is Err) return Err(DomainError.concurrentModification());
    await _audit.append(AuditEntry(/* action: "scheduled" */));
    return Ok(campaign);
  }
}
```

D. Data Structures

`ScheduleCampaignCommand { campaignId: String }` — deliberately minimal; no version parameter is accepted from the caller, because `schedule()` re-derives the version check from the freshly-loaded aggregate itself (see E below), not from anything the UI believes is current.

E. Algorithms and Processing Logic

Operation: Scheduling eligibility check (FR-012, NFR-007)

```
Operation: schedule(campaign)
Input: campaign (Campaign, freshly loaded)
Precondition: none assumed — all checks are performed inside this operation
Steps:
  1. if campaign.state != Approved: raise InvalidTransitionError
  2. if campaign.activeApproval == null
     or campaign.activeApproval.versionReviewed != campaign.currentVersion:
       raise InvalidTransitionError('Approved version is no longer current')
     // this re-check is deliberate: it is NOT assumed from the time of approval,
     // it is re-verified at schedule time, closing the gap a time-of-check/time-of-use bug
         would leave
  3. campaign.state = Scheduled
Output: campaign (mutated)
Postcondition: campaign is Scheduled only if steps 1-2 both passed against the CURRENT aggregate
    state
Error conditions: InvalidTransitionError (covers both "not approved" and "approved version
    stale")
Complexity: O(1)
```

F. Module Interfaces

Interface	Method	Parameters	Return	Exceptions	Calling Module	Consuming Module
`ScheduleCampaign`	`execute(ScheduleCampaignCommand)`	command	`Future<Result<Campaign, DomainError>>`	none (typed `Result`)	Presentation	—

Table 27

G. Database Specifications

No new table concept — writes only the state transition and its audit entry, using the same `campaigns`/`audit_log` concepts as 5.2.3.G/5.2.5.G.

H. Error Specifications

Error ID	Description	Trigger	User Message
ERR-SCHED-01	`InvalidTransitionError` (not approved)	`state != Approved`	"Only approved campaigns can be scheduled."

Error ID	Description	Trigger	User Message
ERR-SCHED-02	`InvalidTransitionError` (stale approval)	Approved version no longer current	"This campaign was modified after approval and must be re-approved before scheduling."
ERR-SCHED-03	`ConcurrentModificationError`	Two concurrent schedule requests	Second request rejected — campaign no longer in Approved state by the time it writes

Table 28

I. Input/Output Specifications

Scenario	Input	Output
Valid schedule	`state == Approved`, approved version == current version	`Ok(Campaign)`, state = Scheduled
Duplicate schedule	Second concurrent request, first already succeeded	`Err` — campaign no longer Approved
Stale approval	Campaign edited after approval without re-review (should not occur if 5.2.2's material-change path is followed, but checked anyway)	`Err(InvalidTransitionError)`

Table 29

J. Dependencies

Internal: `Campaign` aggregate, `CampaignRepository`, `AuditLogRepository`, `AuthProvider`. External: none (actual delivery/scheduling execution is out of scope).

K. Test Specifications

Test ID	Scenario	Expected Result
TC-SCHED-01	Valid schedule	`Ok`, state = Scheduled
TC-SCHED-02	Duplicate concurrent schedule	Second request fails — state already advanced
TC-SCHED-03	Schedule a non-Approved campaign	`Err(InvalidTransitionError)`

Table 30

---

6. Critical Modules — Deep Dive

6.1 Campaign State Management

States: exactly six, per the approved functional requirements: `Draft`, `Pending Review`, `Changes Requested`, `Approved`, `Rejected`, `Scheduled`.

Formal Transition Table

From	Command	Event Emitted	Guard	To	Authorized Actor
Draft	`SubmitCampaign`	`CampaignSubmitted`	Required fields complete (FR-001); creates `CampaignVersion`	Pending Review	Marketer
Pending Review	`ReviewCampaign(approve)`	`CampaignApproved`	Not self-review; `versionBeingReviewed == currentVersion`	Approved	Reviewer
Pending Review	`ReviewCampaign(reject)`	`CampaignRejected`	Same, + `reason` non-empty	Rejected	Reviewer
Pending Review	`ReviewCampaign(requestChanges)`	`ChangesRequested`	Same, + `feedback` non-empty	Changes Requested	Reviewer
Pending Review	`WithdrawSubmission`	`SubmissionWithdrawn`	No `ApprovalRecord` yet for current version	Draft	Marketer
Changes Requested	`ReviseCampaign`	`CampaignRevisedAndResubmitted`	Creates new `CampaignVersion`	Pending Review	Marketer
Approved	`ScheduleCampaign`	`CampaignScheduled`	Approved version still current, re-checked at schedule time	Scheduled	Marketer
Approved	*(system-triggered on edit attempt)*	`ApprovalInvalidated`	Any edit (FR-014)	Draft	System
Rejected	`RestartCampaign`	`CampaignRestarted`	None beyond state check	Draft	Marketer
Scheduled	—	—	Terminal	—	—

Table 31

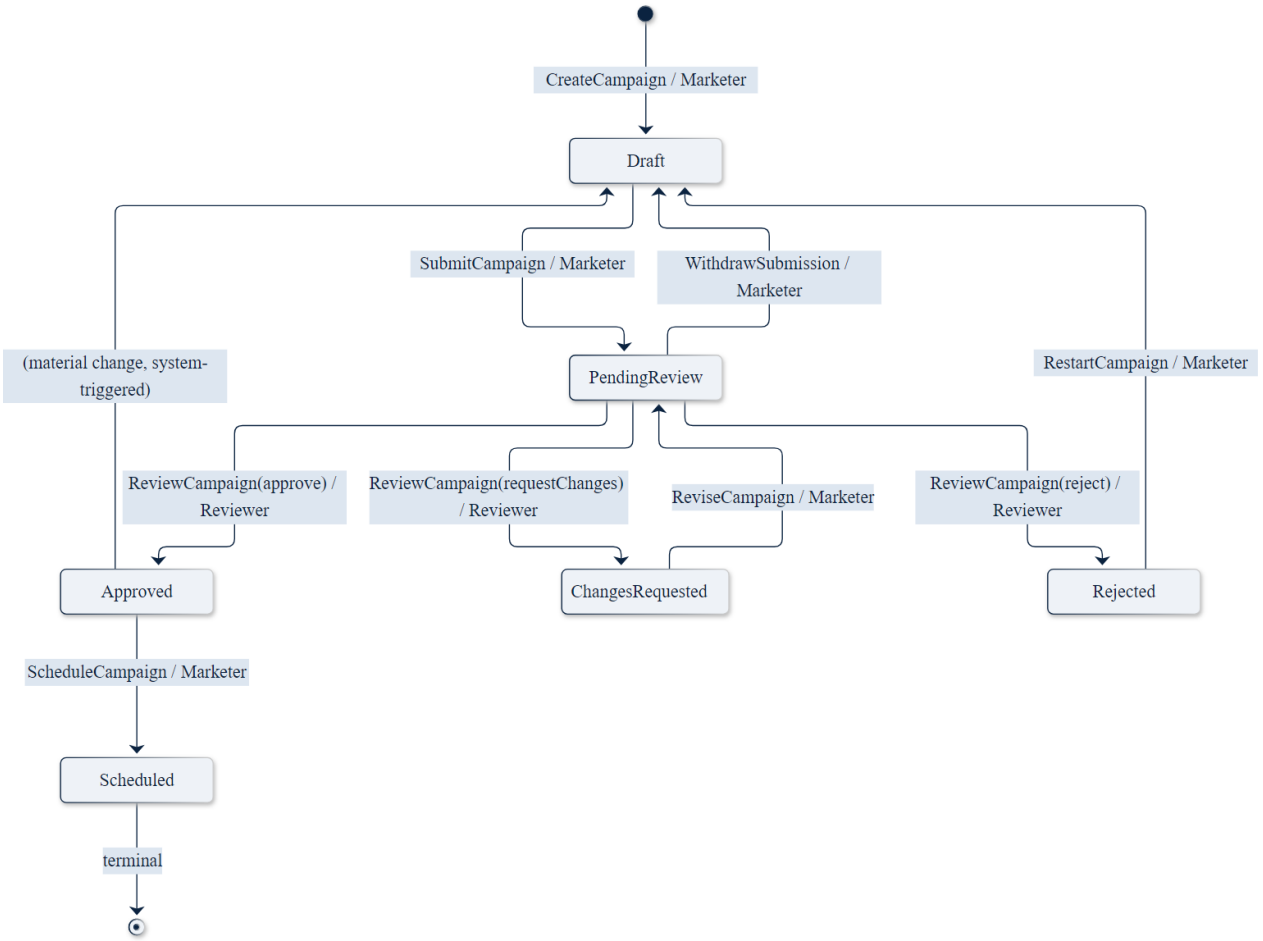


Figure 4: *Campaign State Machine Diagram*

Invalid transitions (illustrative): Draft→Approved, Draft→Scheduled, Pending Review→Scheduled, Scheduled→*(any)*, Rejected→Approved directly — all rejected via `InvalidTransitionError` inside the aggregate (5.2.4.C), never reached because of UI button-hiding alone.

Reconciling the stale-approval scenario with FR-002 (campaigns in Pending Review cannot be edited directly) — only reachable via withdraw-first:

```
Draft --SubmitCampaign--> PendingReview (v1)
                                |
                [Reviewer opens v1, no decision yet]
                                |
PendingReview --WithdrawSubmission--> Draft
Draft --SubmitCampaign--> PendingReview (v2)
                                |
   [Reviewer, unaware, attempts ReviewCampaign(approve, versionBeingReviewed: 1)]
                                |
                Guard fails: 1 != 2 -> StaleReviewError
```

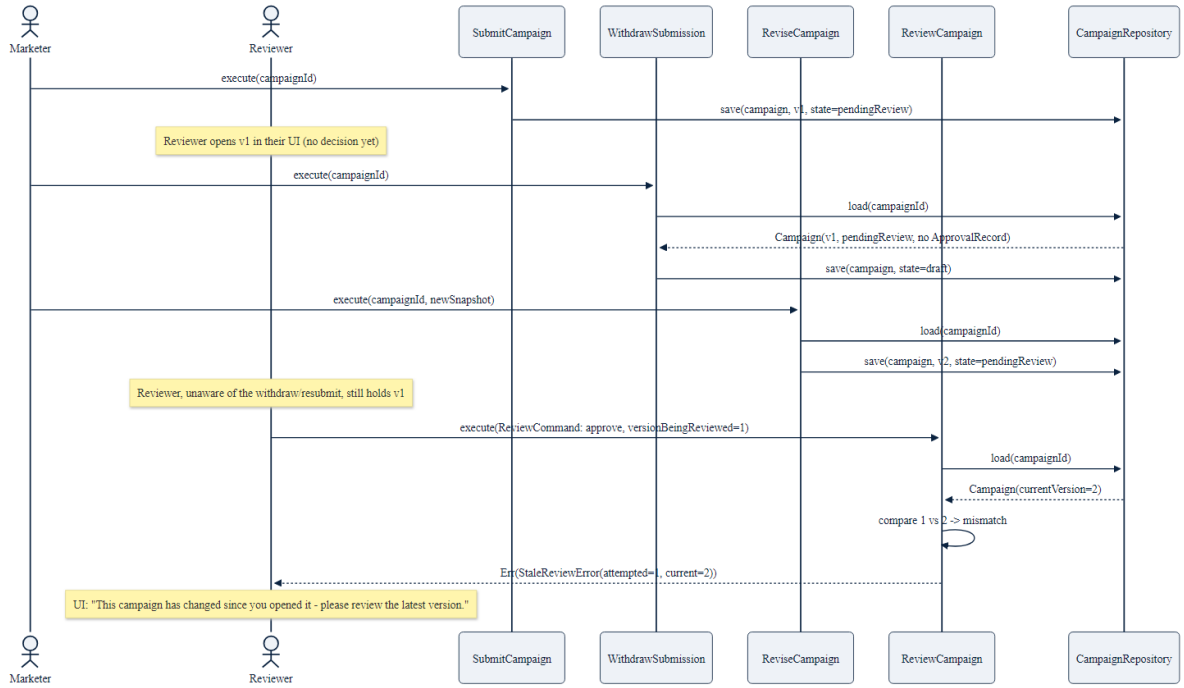


Figure 5: *Campaign Revision / Version-Conflict Sequence*

6.2 Campaign Version Management

Version creation is exactly two trigger points (FR-013): submission and revision-resubmission — never a plain Draft edit (which mutates an unversioned working copy in place). Pseudocode: §5.2.3.E. Material-change invalidation pseudocode:

```
function editApprovedCampaign(campaign, proposedEdit):
    assert campaign.state == Approved
    campaign = campaign.invalidateApprovalOnMaterialChange()  // marks approval superseded,
        -> Draft
    campaign = applyEdit(campaign, proposedEdit)               // edit applied only AFTER
        invalidation
    return campaign
    // Two sequential, independently auditable steps -- never collapsed into one opaque
        operation.
```

6.3 Approval Workflow

Approval command, reviewer authorization, version validation, atomic persistence, audit recording, duplicate-action protection: all specified in §5.2.4 (class, algorithm, error, and test specs). **Transaction boundary:** exactly one local Drift transaction per `ReviewCampaign.execute()` call, covering the campaign-state update, the new `ApprovalRecord` write (implicit in the campaign row's embedded state or a joined row, depending on the final physical schema), and the `AuditEntry` insert — all three commit together or not at all.

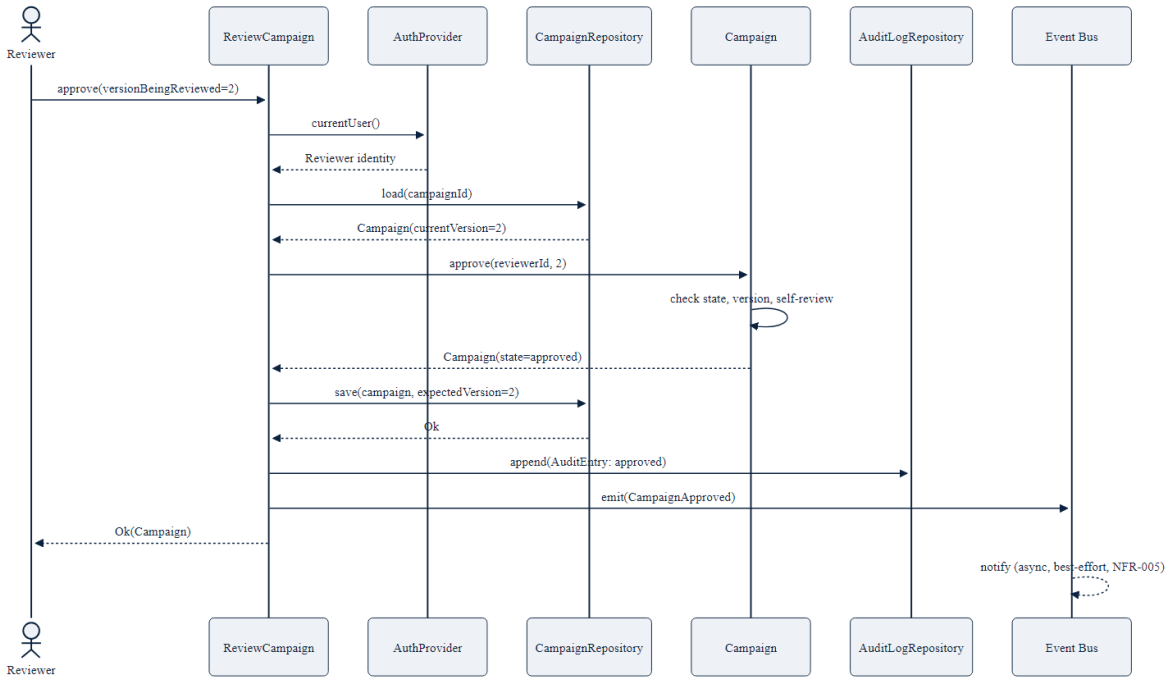


Figure 6: *Approval Transaction Sequence*

6.4 Concurrent Modification

Strategy: optimistic concurrency, justified by NFR-001/003 and the single-device, low-write-contention scale (NFR-015) — pessimistic locking rejected as disproportionate (full rationale: `MAL4-HLD.md` §4.6, §4.14).

The concurrency token: `Campaign.currentVersion` serves two roles — the business-meaningful version number and the optimistic-concurrency token compared at write time. Deliberately not a separate, purely-technical row-version field.

Two distinct checks, two distinct errors:

Check	When	Compares	Error	User-Facing Meaning
Stale-review check	Inside `Campaign.approve/reject/requestChanges`	Caller's `versionBeingReviewed` vs. the aggregate's own `currentVersion`	`StaleReviewError`	"You're reviewing an outdated version"
Optimistic-write check	Inside `CampaignRepository.save()`	The `expectedVersion` loaded at use-case start vs. the version currently stored	`ConcurrentModificationError`	"This campaign changed since you loaded it"

Table 32

Concurrency pseudocode (exact behavior for concurrent modifications):

```
Actor A: load campaign (currentVersion = 1)
Actor B: load campaign (currentVersion = 1)
Actor A: campaign.submit(...) -> currentVersion = 2 (in-memory)
Actor A: repository.save(campaign, expectedVersion: 1) -> succeeds; stored version becomes 2
Actor B: campaign.approve(...) -> currentVersion still 1 in B's in-memory copy (stale)
```

```
Actor B: repository.save(campaign, expectedVersion: 1) -> FAILS: stored version is 2, not 1
        -> ConcurrentModificationError returned to Actor B's use case
Actor B's UI: "This campaign changed since you loaded it -- please reload."
```

No merge is attempted — the second writer's change is rejected outright, never silently dropped or merged.

Duplicate command protection gap, stated honestly: this mechanism only protects against duplicates detected *after* a successful first write. A true race (not a retry-after-success) is not fully addressed — a proper idempotency-key scheme (NFR-004) would close that gap and is **not yet designed**.

Test cases: TC-REV-02, TC-VER-03, TC-SCHED-02 (§5.2 above) cover this directly.

6.5 Audit Management

Full event structure, creation rules, actor identity, action taxonomy, and persistence guarantees: §5.2.5. Correlation identifiers: `campaignId` + `version` together serve as the correlation key across an `AuditEntry`'s related `CampaignVersion`/`ApprovalRecord` — no separate trace/correlation ID scheme exists beyond this, since there is no distributed call chain to correlate across (single-process, local-first architecture).

6.6 Scheduling

Eligibility checks, approval validation, version consistency, scheduling preconditions, duplicate-scheduling protection: §5.2.7, especially the re-verification-at-schedule-time detail in §5.2.7.E (not assumed from the time of approval).

---

7. API and Contract Specifications

No remote backend exists for this deliverable (see `MAL4-HLD.md` §4.15 — no server process is deployed or planned). Accordingly, this section documents the **internal repository and use-case interfaces** rather than a remote API — these are the effective contract surface of the application:

```
CampaignRepository.load(campaignId: String) -> Future<Campaign>
CampaignRepository.save(campaign: Campaign, {required int expectedVersion}) ->
    Future<Result<void, ConcurrentModificationError>>

AuditLogRepository.append(entry: AuditEntry) -> Future<void>
AuditLogRepository.queryBy({String? campaignId, String? actor, DateTimeRange? timeRange}) ->
    Future<List<AuditEntry>>

CreateCampaign.execute(CreateCampaignCommand) -> Future<Result<Campaign, DomainError>>
EditCampaign.execute(EditCampaignCommand) -> Future<Result<Campaign, DomainError>>
SubmitCampaign.execute(SubmitCampaignCommand) -> Future<Result<Campaign, DomainError>>
WithdrawSubmission.execute(WithdrawCommand) -> Future<Result<Campaign, DomainError>>
ReviewCampaign.execute(ReviewCommand) -> Future<Result<Campaign, DomainError>>
ReviseCampaign.execute(ReviseCampaignCommand) -> Future<Result<Campaign, DomainError>>
ScheduleCampaign.execute(ScheduleCampaignCommand) -> Future<Result<Campaign, DomainError>>
```

Every use case returns a `Result` rather than throwing for expected business failures — this *is* the contract's "status code" equivalent: `Ok` ~ 2xx, `Err(UnauthorizedError)` ~ 401/403, `Err(ConcurrentModificationError)` ~ 409 Conflict, `Err(InvalidTransitionError)`/`Err(StaleReviewError)`/`Err(SelfReviewError)` ~ 422 Unprocessable, by rough HTTP-status analogy — stated for reviewer intuition only, since

no actual HTTP layer exists.

Idempotency: per §6.4's stated gap, no formal idempotency-key mechanism exists yet for these interfaces — flagged again here rather than assumed solved.

Version/concurrency requirements: every mutating call that operates on an existing `Campaign` requires an `expectedVersion` (at the repository boundary) and, for review decisions, a `versionBeingReviewed` (at the use-case boundary) — both detailed in §6.4.

---

8. LLD Diagrams

Figure	Diagram	Purpose	Section
Figure 1	Conceptual Entity Relationship Diagram	Entity relationships only — physical schema is defined separately in the project's data-design documentation	§5.2.3
Figure 2	UML Class Diagram	`Campaign` aggregate, value objects, repository interfaces, and `ReviewCampaign`'s dependencies	§5.2.4
Figure 3	Error Recovery Sequence	State commits, notification fails — NFR-005's isolation guarantee, shown as a sequence	§5.2.6
Figure 4	Campaign State Machine Diagram	Full transition table rendered as a state diagram	§6.1
Figure 5	Campaign Revision / Version-Conflict Sequence	The reconciled stale-approval scenario, end to end	§6.1
Figure 6	Approval Transaction Sequence	Full `ReviewCampaign.execute()` call sequence, including the atomic save and the post-commit event emission	§6.3

Table 33
